



Large Language Model-Enabled Multimodal UAV for Rescue

Allan Dong*, Raymond Trinh, Xinwei Choot, Benjamin Seow
The University of Sydney, Sydney, NSW, 2008, Australia

This paper presents LEMUR (Large Language Model-Enabled Multimodal UAV for Rescue), an initial framework to integrate LLM-based planning with UAV control. LEMUR uses Gemini 2.5 Flash from Google as a high-level mission planner, and couples it with low-level drone motion commands and vision-based perception modules through the Google Agent Development Kit (ADK). We evaluated LEMUR's performance in traversing courses against a state machine using standardised obstacles. Our results show that using an LLM to plan a trajectory for missions is viable, and presents advantages over the state machine, such as its flexibility in traversing novel environments. However, in some missions, it exhibited slower behaviour, especially when decisions from feedback are required.

I. Introduction

As students, we are witnessing breakthroughs in artificial intelligence (AI) at an astonishing pace, and the notion of AI replacing human capabilities no longer seems distant. Social media reinforces this narrative, with technology leaders heralding AI as rivals to human intelligence or even the benchmark of superhuman performance. For graduates entering robotics and autonomous systems industry today, the line between AI as a tool for humans, and AI as a replacement entirely, has become increasingly blurred. Amidst so many narratives surrounding AI capabilities, we seek to answer a fundamental question: what is the *actual* state of AI capability in real-world physical tasks?

The concept of *intelligence* is notoriously difficult to define technically. A growing body of research suggests it cannot be separated from physical interaction with the environment, a concept known as "embodied cognition" [1]. This has profound implications for autonomous systems: if intelligence emerges from the coupling of cognitive processes with sensorimotor capabilities, then advances in artificial intelligence and robotic control should be combined rather than developed in isolation.

Recent breakthroughs in large language models demonstrate sophisticated reasoning and planning capabilities, while parallel developments in drone hardware have improved sensing and actuation. We hypothesise that integrating these two streams, LLM-based planning with UAV sensorimotor control, can enable autonomous systems to complete complex, multi-phase missions that currently require either constant human supervision or extensive preprogrammed logic tailored to specific scenarios.

Consider a scenario where a hiker becomes lost in remote bushland and a UAV search-and-rescue mission is deployed to locate this hiker. In this multi-phase mission UAV must navigate through unknown obstacles, execute an efficient search pattern to locate the missing person, capture identifying information, and return to base, all under limited sensing capabilities and constrained battery endurance. A single human operator is bandwidth-limited and unable to continuously pilot while processing sensor data and making strategic decisions. A purely automated pipeline constrained by fixed state machines and preprogrammed responses lacks the flexibility to adapt when environmental conditions deviate from predefined scenarios. Today, reinforcement learning and imitation learning have demonstrated effectiveness in teaching robots to perform complex tasks through interaction and demonstration. However, these approaches often struggle with adaptation to novel environments and scenarios not represented in training data.

Recent advances in large language models (LLMs) offer potential solutions to these challenges. The process of human mission planning, breaking down abstract objectives into executable sub-tasks while maintaining contextual awareness, is a type of cognitive process that LLMs can approximate artificially through their advanced contextual understanding and generalisation abilities [2].

A growing body of research has explored LLM integration for robotic tasks. VoxPoser [3] utilises LLMs to perform everyday manipulation tasks through 3D value maps, while Robotics Transformer (RT-1) [4] leverages large-scale learning data to enable task execution beyond training scenarios. Code-as-Policies [5] demonstrates that LLMs can

*Contact: Adon0970@uni.sydney.edu.au
AIAA Membership Category: Student Member
Student Category: Undergraduate Team
Membership Numbers: 1861884, 1863749, 1859216, 1584189

generate executable robot programs from natural language. However, challenges remain in effectively integrating LLMs into autonomous aerial systems. These include complex prompting requirements, limited real-time feedback integration, and inefficient pipelines that hinder seamless mission execution. Moreover, current approaches have not fully explored tool-augmented generation architectures, where LLMs can dynamically select from libraries of control primitives to execute a mission. This approach has the potential to enable flexible mission adaptation without retraining.

There is a need for an innovative approach in autonomous UAV control that combines the latest developments in AI with robust sensorimotor feedback to execute missions effectively in the face of uncertainty. Large language model-enabled multimodal UAV for rescue (LEMUR) is a framework that integrates LLM-based planning with multirotor drone control in a novel search-and-rescue mission architecture. Its usefulness arises from coupling vision-based perception with the cognitive planning capabilities provided by an integrated LLM agent that selects from a curated library of motion control primitives. We hypothesised that LEMUR would allow a UAV to complete a representative search-and-rescue mission, navigating obstacles, conducting search, locating a target, and returning to base. To validate this hypothesis and assess practical viability, we implemented both LEMUR and a basic finite state machine on identical hardware, testing both systems on a standardized indoor course simulating a search-and-rescue mission.

LEMUR employs Google’s Gemini 2.5 Flash [6] as a high-level mission planner, coupled with the DJI Tello SDK that wraps low-level flight dynamics control in natural-language-accessible tool functions. The agent receives mission objectives in plain language, dynamically generates action sequences by selecting appropriate tools, and adapts its plan based on structured feedback from each executed action. This approach encodes mission knowledge not through explicit state transitions, but through system instructions that guide the LLM’s planning process and a library of motion primitives. We hypothesised that the agent could replicate and potentially exceed the capabilities of a preprogrammed state machine (SM) over our fixed evaluation course, while providing greater adaptability to environmental variations. This work represents progress toward scalable, efficient autonomous aerial systems capable of completing complex missions in uncertain environments, a critical capability for practical search-and-rescue deployment.

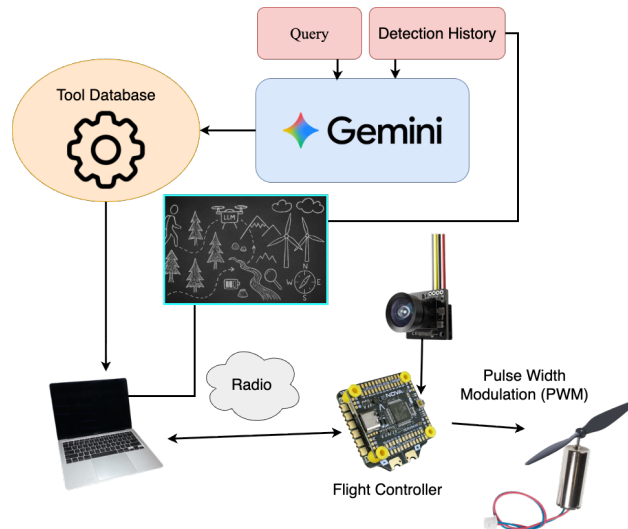


Fig. 1 Schematic of the LEMUR system framework: The schematic illustrates the system architecture, showing the high-level planning layer (Gemini LLM agent) and low-level execution layer (drone hardware and flight controller). User queries and detection history are fed into Gemini 2.5 Flash, which integrates this input with (i) a live video stream from the drone’s onboard camera showing the environment; (ii) a tool database containing motion primitives and perception functions. Gemini decomposes high-level mission objectives into actionable sub-tasks, retrieves relevant tools from the database, and generates structured function calls tailored to current conditions. These commands are transmitted to the ground station (MacBook M1) via WiFi radio communication. The ground station processes commands and sends control signals to the drone’s flight controller, which converts them to motor commands via pulse-width modulation (PWM). Visual feedback from the camera enables object detection (YOLO processing on ground station) and closed-loop control. The tool database returns structured responses (success/failure, environmental data) to Gemini, enabling the agent to assess outcomes and adapt subsequent actions based on real-time feedback.

II. Results and Discussion

A. Abstract Reasoning and Geometric Pattern Execution

To assess the initial performance of the LLM agent, we designed a test to evaluate its ability to interpret abstract instructions and translate them into a sequence of executable code that makes the drone fly in the intended trajectory. The following prompt was provided to the LLM agent: "Design a set of instructions to make the drone fly in a pattern. The pattern will be: star. Take-off and execute the manoeuvres. The width should not exceed 300 cm." This tested the agent's open-loop performance in the absence of feedback from external sources about the drone's position.

The agent was successful in translating the high-level objective provided into a flight plan that reflects the star geometry. It determined the required angles to turn by, and the distances to move forward by the trace out the path of a star while meeting the width constraint. Its flight plan consisted of two phases. The first phase was an initial ascent and positioning where the drone took off and moved forward by 150 cm to reach the first point of the star. The second phase was the tracing out of the star shape where the drone made 5 repetitions of a 144° anti-clockwise turn followed moving forward by 285 cm. The agent predicted that this would then return the drone to its starting position, facing the original direction.

To assess the drone's accuracy, an OptiTrack motion capture system was used to track its actual flight path. Four retroreflective markers were attached to the drone, and position data from the OptiTrack system was captured at 120 Hz. The drone was instructed to fly in the path of a star shape 5 times to assess its reproducibility in following instructions and its drift.

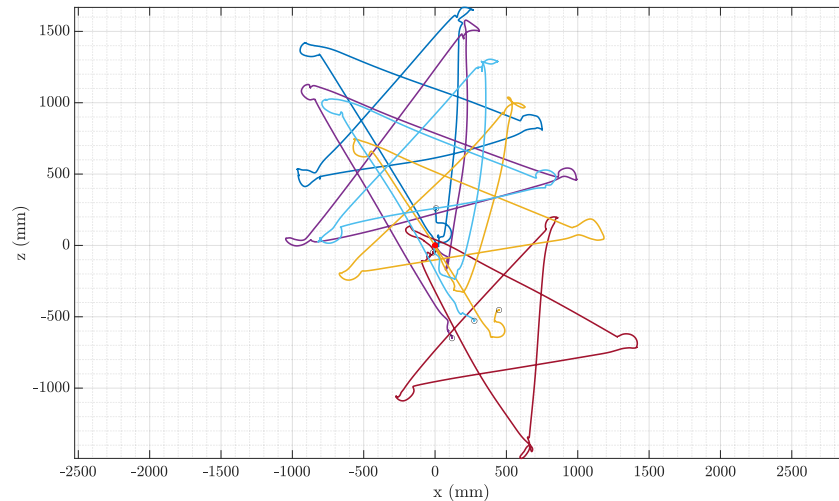


Fig. 2 OptiTrack motion capture of drone flight trajectories when commanded to draw a star pattern. Trajectories have been spatially aligned to share a common starting point at (0,0), indicated by the red point. Final positions of each trajectory are marked with open black circles.

Figure 2 illustrates the five star trajectories flown by the drone. While the agent's geometric reasoning appears to be correct in following the rough shape of a star, there are some significant deviations that can be observed in the trajectory. The final position of the drone after completing the star shape differs on the order of 50 cm, reflecting a heading drift in the drone's flight. This is likely caused by heading errors in the drone's inertial measurement unit (IMU) that accumulate over the course of the flight. Distortions in the path are also notable in the shape of the central region of the star patterns, showing that the star's vertex angles were not exactly 36° . Each section of the star, consisting of a forward movement and a rotation, occurs in an open-loop without any external position reference. With 5 of these sections, so 10 movements in total, small errors in the forward movement and rotation will compound and result in deviations in the observed path.

Another system that was also evaluated was a multi-agent setup where a dedicated "planner" agent was tasked with generating a flight sequence from the human input, while a separate "executor" agent was tasked with executing the movement controls designed by the planner agent. No significant improvement in the drone's performance was observed with this change. It was observed that implementing this structure increased the overall latency time between the human

input being passed to the drone, and the drone executing its movements due to the presence of multiple agents. The plan created by both systems were identical, so any additional reasoning ability is outweighed by the time required for this task. However, for more complex tasks that require sensor feedback, this could offer a significant benefit.

These results confirm how important feedback loops from vision information are in navigation. Using an IMU-only approach can lead to errors being accumulated, which may be detrimental to tasks requiring precision, such as approaching a target and landing. A closed-loop system that adjusts heading and velocity continuously based on sensor information can overcome this limitation. Overall, the LLM agent has demonstrated geometric reasoning abilities, including calculating angles and distances required to make a drone fly in a certain shape. However, a key challenge that will need to be addressed is trying to obtain the accuracy that can be achieved through a closed-loop system.

B. Obstacle Course Navigation

To quantitatively compare the LLM agent with the baseline state machine, 6 standardised obstacle courses were designed which composed of identical vertical poles which were arranged in various configurations. These small poles were selected to maintain reproducibility of results while allowing for flexibility in designing configurations. Each system was assessed based on its completion time over four mission phases:

- Stage 1: System Initialisation (last human interaction → takeoff)
- Stage 2: Perception Activation (takeoff → first rotational motion)
- Stage 3: Navigation Engagement (first rotation → first translation)
- Stage 4: Obstacle Traversal (first translation → landing after clearance)

Figure 3 illustrates the six obstacle course configurations. Each configuration requires different decisions to be made to navigate through.

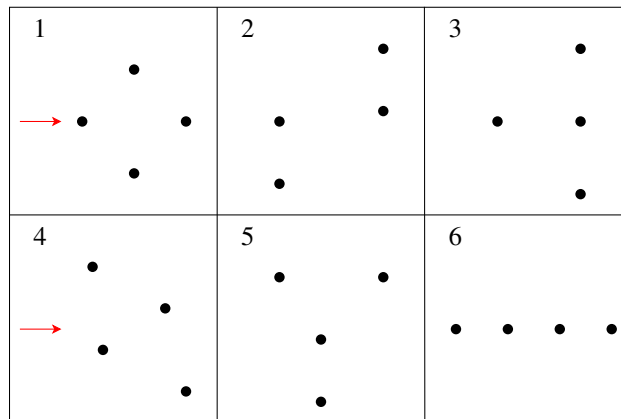


Fig. 3 Pole configurations tested (red arrow indicates direction of travel)

1. Stage Timing Analysis

Table 1 presents completion times for each system across all courses. Times recorded have $\pm 0.5s$ uncertainty due to reaction time. For the state machine, Stages 2 and 3 have been combined as the first observed motion was translational rather than rotational.

In Stage 1, both the agent and state machine exhibited similar takeoff latencies. This similarity was expected because the tasks involved in this period were establishing the drone connection, starting the camera, and loading the computer vision model, which were the same across both systems. The agent also needed to process the human prompt, but this was performed quickly, only creating variations that lay within the measurement uncertainty.

In Stage 2, the state machine consistently moved forward first, faster than the agent was able to complete its first rotation and translation. The state machine's control loop was able to nearly immediately analyse the incoming frames and compute the required commands to navigate, resulting in its quick first translation times of 4.4 - 6.6 seconds. The agent initiated rotational motion after 8.1 - 12.9 seconds, while its first translational motion occurred 4.3 - 5.2 seconds later. The agent's slower time reflects the greater time required to call the correct tools. From the logging, it starts the video stream, waits for confirmation, initiates YOLO detections, waits for them to accumulate, then decides which

Table 1 Stage-wise and total times (seconds) for State Machine vs Agent.

System	Course	Stage 1 (s)	Stage 2 (s)	Stage 3 (s)	Stage 4 (s)	Total (s)
State Machine	1	2.76	–	4.42	13.05	22.99
	2	2.61	–	6.60	22.85	34.67
	3	2.51	–	5.71	20.77	31.50
	4	2.58	–	5.95	8.38	19.49
	5	2.60	–	6.29	15.12	26.61
	6	2.74	–	5.48	31.12	42.08
Mean $\pm$ SD		2.63 $\pm$ 0.09	–	5.74 $\pm$ 0.58	18.38 $\pm$ 8.17	29.89 $\pm$ 8.04
Agent	1	2.68	10.97	5.10	33.49	52.24
	2	2.61	12.02	5.13	53.32	73.08
	3	2.38	12.02	5.21	36.23	55.84
	4	2.91	11.22	5.11	34.66	53.90
	5	2.66	12.85	4.34	21.23	41.08
	6	2.34	8.14	4.51	59.39	74.38
Mean $\pm$ SD		2.60 $\pm$ 0.20	11.20 $\pm$ 1.58	4.90 $\pm$ 0.34	41.39 $\pm$ 13.40	58.75 $\pm$ 12.52

direction to move. Each LLM call adds latency, resulting in the longer stage time, while the state machine was able to activate its sensors quickly to begin moving.

There was the greatest divergence in performance between the two systems in Stage 4, with the highest variation in course completion time. The completion time of the state machine ranged from 8.4 - 31.1 seconds, while the completion time of the agent ranged from 21.2 to 59.4 seconds. It was observed visually that the agent spent about 40% of its Stage 4 time hovering as it waited for the LLM to construct a navigation plan. This is a result of the structure where in each decision, a set of actions are executed, then control is returned to the LLM to plan the next command. This results in large periods of inactivity where the drone was stationary, increasing the course completion time.

As the state machine operated using a closed feedback loop system, it was able to stay in constant motion, reducing its completion time. The system implemented was structured around detection of the poles, where when poles were detected, gap detection was performed, which was then fed into a PID controller to decide the appropriate movements. This different structure resulted in the state machine's superior performance in Stage 4, even though both systems had similar low level control and perception algorithms. Course 6 required the longest completion time for both the agent and the state machine due to its layout, where the drone completed a weaving motion between the poles.

C. Search and Rescue (SAR)

The next phase of the mission consists of locating the missing person within a designated search radius. This phase evaluates the system's ability to switch from navigation to target acquisition using a systematic search pattern, then precisely locating the target. Three trials were conducted for each system, with the time taken for the drone to pass through the following stages recorded.

- Stage 1: System Initialisation (last human interaction $\rightarrow$ takeoff)
- Stage 2: Search zone approach (takeoff $\rightarrow$ first movement toward search area)
- Stage 3: Target acquisition (first movement $\rightarrow$ person detected)
- Stage 4: Target confirmation (person detected $\rightarrow$ person centred)
- Stage 5: Documentation (person centred $\rightarrow$ confirmation of completion)

Table 2 Search and Rescue completion times per stage for AI (LLM agent) and SM (state machine)

System	Trial	Stage 1 (s)	Stage 2 (s)	Stage 3 (s)	Stage 4 (s)	Stage 5 (s)	Total (s)
SM	1	2.42	2.93	8.26	-	58.70	72.31
	2	2.77	2.05	8.12	-	58.14	71.08
	3	2.50	3.73	11.50	-	48.92	66.65
	Mean $\pm$ SD	2.56$\pm$0.18	2.90$\pm$0.84	9.29$\pm$1.89	-	55.25$\pm$5.50	70.01$\pm$2.86
Agent	1	2.63	10.91	9.80	4.61	9.68	37.63
	2	2.63	17.97	22.57	10.80	8.13	62.10
	3	2.68	11.77	44.96	-	3.85	63.26
	Mean $\pm$ SD	2.65$\pm$0.03	13.55$\pm$3.90	25.78$\pm$17.83	7.71$\pm$3.10	7.22$\pm$2.92	54.33$\pm$14.67

1. Stage Timing

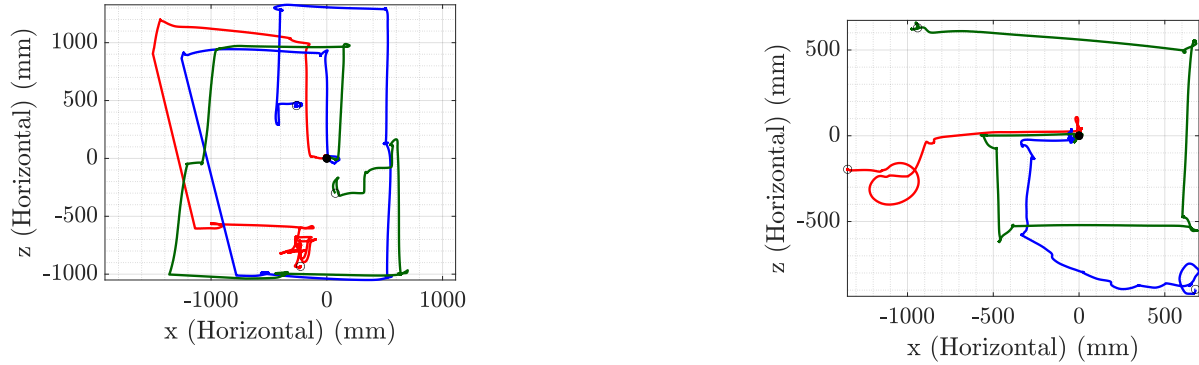


Fig. 4 OptiTrack motion capture of drone flight trajectories for Search and Rescue mission under state machine (left) and LLM agent (right). Trajectories have been spatially aligned to share a common starting point at (0,0), indicated by the solid black point. Final positions of each trajectory are marked with open black circles.

2. Stage Timing Analysis

The Stage 1 times for both the SAR and obstacle navigation task are aligned. This outcome is expected because the structures of both the agent and state machine have not changed significantly in the initialisation phase. In Stage 2, the state machine initiated movement toward the search area significantly faster, a 4.7 times advantage explained by the agent's sequential tool-calling overhead requiring multiple LLM inference cycles. Notably, the agent exhibited high variability with Trial 2 requiring 17.97s. This suggests intermittent planning complexity. It was observed that the LLM generated a more complex set of instructions which took longer to infer.

In Stage 3, the state machine maintained relatively consistent times while the agent exhibited large variability. In Trial 3, the agent took 44.96s, which is 4.8 times longer than its best trial and 3.9 times longer than the state machine mean which indicates severe sensitivity to the location of the person. In particular, the state machine was set up with the same search pattern which ensured that all of the search radius was covered whilst the agent generated simpler, more generalisable patterns such as squares and spirals of increasing increment.

For the target confirmation and documentation, Stage 4 and 5 are completed in one go for the state machine whereas they are distinct steps for the agent. However, the sum of the stages for the agent is still far lower than the state machine which results in similar final times apart from the outlier of the agent in Trial 1. The main reason for this is because the state machine had a much tighter tolerance for centring whereas the agent decided that as long as the target was in frame, then it was satisfactory to mark as centred. In practice, the state machine has the benefit of a closer shot which enables

more information to be gathered on the health of the lost individual while the agent prioritises speed in successfully locating the person.

D. Return to Base

The final mission phase involves an autonomous return to the launch pad after completing search operations. It tests each system's ability to navigate back to a known location and execute precision landing. This phase is critical for operational SAR missions where battery conservation requires efficient return trajectories and accurate terminal guidance. Both systems were evaluated across three trials using OptiTrack motion capture to record ground-truth flight paths. Mission timing was segmented into five stages:

- Stage 1: System Initialisation (last human interaction → takeoff)
- Stage 2: Perception Activation (takeoff → first rotational motion)
- Stage 3: Navigation Engagement (first rotation → first translation)
- Stage 4: Travelling to Base (first translation → located QR code)
- Stage 5: Final descent and landing (located QR code → landing)

1. Stage Timing

Table 3 Return-to-base completion times per stage for AI (LLM agent) and SM (state machine). Uncertainty on manual timing is approximately ± 0.5 s.

System	Trial	Stage 1 (s)	Stage 2 (s)	Stage 3 (s)	Stage 4 (s)	Stage 5 (s)	Total (s)
SM	1	4.45	6.95	5.77	17.51	80.24	114.92
	2	2.80	6.00	5.45	18.02	6.72	38.99
	3	2.80	11.19	5.53	19.07	69.22	107.81
	Mean	3.35 ± 0.95	8.05 ± 2.86	5.58 ± 0.18	18.20 ± 0.78	52.06 ± 41.13	87.24 ± 42.59
Agent	1	2.51	13.45	16.40	28.04	4.15	64.55
	2	2.35	13.90	12.95	30.44	6.30	65.94
	3	2.46	11.78	25.83	39.44	10.55	90.06
	Mean	2.44 ± 0.08	13.04 ± 1.13	18.39 ± 6.73	32.64 ± 5.18	7.00 ± 3.23	73.52 ± 14.32

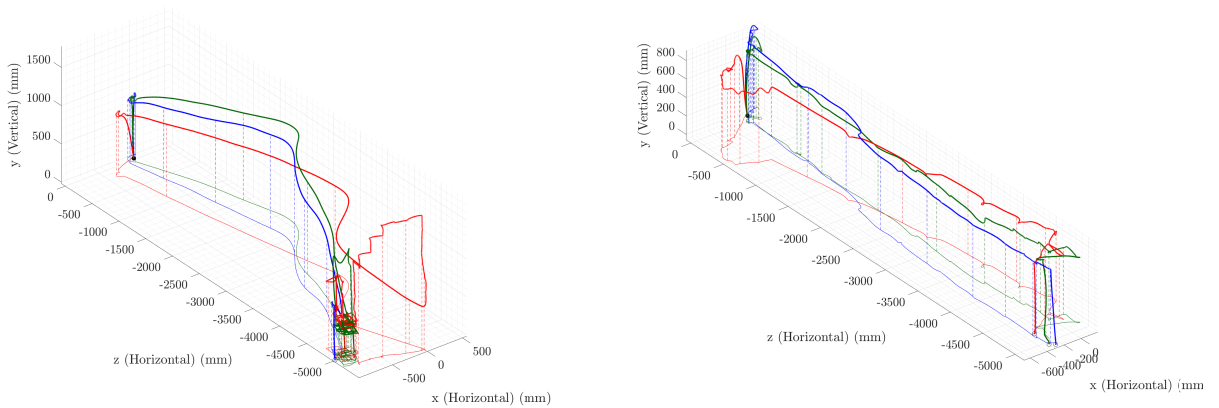


Fig. 5 OptiTrack motion capture of drone flight trajectories for Return to Base mission under the state machine (left) and LLM agent (right). Trajectories have been spatially aligned to share a common starting point at (0,0), indicated by the solid black point. Final positions of each trajectory are marked with open black circles.

2. Stage Timing Analysis

In Stage 1, the agent was able to commence movements faster. During the initialisation phase, the state machine turns frame streaming and YOLO perception on before taking off. On the contrary, the agent takes off first then starts the camera systems. The faster takeoff time for the agent reflects this difference.

However, in Stage 2, the state machine was faster than the agent. This is expected as the state machine already has its cameras on at this point and is not required to plan out what actions it will take after taking off. The agent was slower because it also needed to do this, while also taking the time to think of and plan the next action.

In Stage 2, the agent was slightly faster than the state machine. Both systems execute rotations to search for the large QR code placed near the landing pad, so similar times would be expected. The state machine's higher variability indicates inconsistent detection. For example, Trial 3 required 11.19s (4 extra rotations), possibly due to lighting changes or reflections from the glass wall mentioned in the baseline system documentation.

The state machine was able to complete Stage 3 faster than the agent, reflecting how it started its translational motion soon after its first rotational motion. The key reason why this occurred was that the drone with the state machine was able to continuously rotate around while monitoring for the QR code due to its feedback structure. This allowed it to quickly locate the QR code and commence translating. The agent's lack of a continuous feedback loop meant it had to alternate between detecting the QR code and rotating, with each cycle taking up time in between as the agent thinks about what the next move should be.

Similarly, the performance in Stage 4 was faster for the state machine in general. This occurred for the same reason as stated for Stage 3, where the continuous feedback loop allows the drone to move forward while making detections. The agent again moved in steps, slowing it down while it calculated the next move. However, it compensated for this by being able to fly forward significant in large straight sections, which are clearly visible in Fig 5, while the state machine was slower and followed a more curved path as it continually adjusted its motion from detections.

Finally, the most dramatic difference was observed in Stage 5. The agent maintained consistent performance, but the state machine showed extreme variability. This is because the state machine descends in small increments to ensure the final position is precisely within the landing zone whereas the agent takes larger steps with higher tolerance. This is reflected in messy paths in Fig 5 in the left diagram, while the agent is able to execute a smooth descent to the base.

E. Overall Performance

To assess end-to-end mission capability, we analysed total completion times across the complete search-and-rescue scenario by summing the three sections: obstacle navigation to search zone location to target acquisition and then finally, return to base. For the three full mission trials, the LLM agent achieved faster mean mission completion times, corresponding to time savings over the state machine baseline. This represents a significant performance advantage for the complete mission profile, despite the agent's demonstrated $2 \times$ penalty during isolated obstacle navigation phases. The total results reveal that mission-level performance does not simply reflect the sum of individual phase characteristics. While the state machine dominated obstacle traversal, the agent's advantages in search operations and return navigation compounded to produce superior overall performance.

This demonstrates that for multi-phase missions, the critical metric is end-to-end completion time rather than performance on any single sub-task. The agent's ability to execute efficient search patterns and robust return navigation more than compensates for its obstacle navigation overhead. However, the agent exhibited substantially higher variability indicating less predictable performance. For example, in the Search and Rescue mission, the agent completed Trial 2 and 3 in 62-63s, while it completed Trial 1 in 38s. This variability stems primarily from the search phase, where the completion time depended on how well the drone could rapidly locate the target location and commence landing. The state machine's consistent 66-72s range (6s spread) reflects its deterministic execution. Fixed rectangular search patterns and preprogrammed return sequences produce predictable timing regardless of target location. For operational SAR deployment, this raises a fundamental question on whether the guaranteed completion over a longer time with low uncertainties is preferable to variable performance over a potentially shorter time with high uncertainties.

III. Methods

A. Overview

The goal of the autonomous drone system is to navigate complex indoor environments, locate target objects, and identify nearby persons in search-and-rescue scenarios. We designed a realistic setting featuring obstacles including

vertical poles, varied lighting conditions, and occluded target objects. The scenario was designed to test the drone's ability to perform diverse navigation and detection tasks in a realistic, though reasonably constrained, environment while interacting autonomously based on high-level human commands.

We assumed that low-level flight control mechanisms managed basic stabilisation and motor commands. The pipeline consisted of a language-processing component for task decomposition and planning, a vision system for object detection and pose estimation, and a motion execution module for obstacle navigation and target approach. All components were integrated within a Python-based control architecture using the DJI Tello drone platform.

B. Hardware

1. UAV Platform

The DJI Tello Talent drone served as the aerial platform for all experiments. This lightweight (87g) quadrotor provides dual camera systems: a forward-facing 720p color camera (82.6° FOV) for obstacle detection and target approach, and a downward-facing grayscale IR-sensitive camera for overhead positioning and search operations. The drone communicates via 2.4 GHz WiFi with a maximum operational range of 100m and flight endurance of approximately 13 minutes. Unlike GPS-enabled platforms, the Tello lacks onboard positioning sensors, necessitating vision-based localisation throughout the mission.

2. Ground Station

All computation-including LLM inference, YOLO object detection, and control signal generation-was performed off-board on a MacBook M1 (2020) ground station. The Apple Silicon architecture provided sufficient performance for real-time processing: YOLO inference at 15-20 FPS (640×640 input resolution) and LLM tool selection latency of 300-600ms per decision. The ground station maintained continuous WiFi communication with the drone, receiving live video streams (720p @ 30fps) and transmitting control commands at 10Hz. This architecture mirrors the Tello's design constraints, as it lacks onboard processing capabilities for vision or planning algorithms.

C. System Architecture Comparison

To evaluate the effectiveness of LLM-based planning for autonomous drone missions, we developed and compared two distinct system architectures: a traditional finite state machine (baseline) and an LLM-based agent (proposed approach). Both systems were implemented on identical hardware and evaluated on the same search-and-rescue mission scenario.

1. Baseline: State Machine Implementation

The baseline system employs a finite state machine with eight discrete operational states. Within each state, the drone has access to different Python functions which it executes in a pre-defined order. State transitions are governed by hard-coded conditional logic checking the completion criteria of each state. The state definitions and logic in order of execution are:

- **Initialisation:** Loads the trained YOLO model, establishes WiFi connection to the Tello, initialises the forward camera stream, and executes takeoff.
- **Navigate Poles:** Applies YOLO detection on each frame to identify poles with a confidence of greater than 50%. Stores bounding box centres and calculates the largest horizontal gap between consecutive poles using a 20-frame sliding window for trajectory smoothing. A PID controller aligns the drone to the gap centre, then advances forward until the gap reaches a desired pixel width. Transition occurs when <2 poles are detected in frame, indicating clearance.
- **Search Zone Locating:** Executes 30° clockwise rotational increments while scanning for a physical search marker, such as a QR code, via YOLO. This marker was necessary due to the Tello's unreliable IMU where high drift prevented accurate dead-reckoning navigation. Upon detection, the drone centres the marker in the forward camera frame using PID control.
- **Transition Step:** Executes a timed forward movement of 20 cm/s for 7 seconds to position the drone approximately over the search marker. Switches to downward camera.
- **Search Zone Centering:** Ascends in 20cm increments until the search marker is detected in the downward camera. Uses PID control to center the marker, then descends until the bounding box area exceeds a certain number of

pixels, representing a close distance to the marker.

- **Locate:** Executes a predefined rectangular spiral search pattern with 1m steps. If the person is not detected after one cycle, the drone ascends 30cm to expand its ground coverage and repeats. Upon detecting a person with confidence of greater than 50%, it centres the target within a tolerance, captures a photo, performs a 360° rotation, and transitions to Return.
- **Return:** Rotates in 30° increments searching for a large QR code (placed near landing pad) using QReader. Upon detection, PID control centres the QR code horizontally while moving forward. When the QR code pixel area indicates proximity to landing pad, switches to downward camera.
- **Landing:** Searches for a smaller QR code on the landing pad center. If not visible, ascends in 20cm increments. Once detected, PID control centers the QR code. Descends in 30cm increments until QR code area reaches a size indicating it is close to the ground, then executes the land command.

2. Proposed Approach: LLM-Based Agent

Our LLM-based system replaces the state machine with a Google Gemini 2.5 Flash language model functioning as a high-level mission planner and integrated through Google's Agent Development Kit (ADK). Rather than pre-programming state transitions, the LLM dynamically generates action sequences by selecting from a library of tool functions. The system exposes 18 executable tool functions that wrap and orchestrate Tello SDK operations into high-level, LLM-callable interfaces. Rather than mapping one-to-one to a single SDK primitive, many tools execute sequenced command chains (e.g., combinations of forward x , cw θ , up z , camera switches, and perception/planning calls) with timing, safety checks, and result aggregation.

- **Perception & camera:** `start_streaming`, `stop_streaming`, `start_yolo`, `stop_yolo`, `fetch_latest_detections`, `toggle_camera_direction`, `perception_status`
- **Basic motion:** `takeoff`, `land`, `turn_cw`, `turn_ccw`, `fly_up`, `fly_set_dist_forward`
- **Navigation primitives:** `home_to_target_forward` (vision-guided approach), `center_target_downward` (overhead alignment), `navigate_gap_or_bypass_single_lateral` (dual-mode obstacle handling)
- **Planning/analysis:** `plan_largest_pole_gap`
- **Utilities:** `get_battery`

Each tool returns a structured dictionary indicating success/failure and relevant data (e.g., `"ok": true`, `"gap_px": 245.3`, `"step_cm": 45`), enabling the LLM to evaluate outcomes and adapt subsequent actions.

D. Vision System and Object Detection

A YOLOv8 model (fine-tuned on custom pole and target object datasets) was successfully deployed for the language-to-vision module. The vision system operates in a background thread, continuously processing camera frames at a target frame rate and publishing detection results to a thread-safe blackboard architecture (PerceptionBlackboard). This design allows the LLM to query the latest environmental state asynchronously without blocking navigation operations.

A total of 472 base images were collected from multiple camera sources and annotated using Roboflow. All images were standardized to 640×640 resolution. To improve model robustness, extensive data augmentation was applied including: spatial transformations (0-30% crop zoom, ±15° shear), color variations (±25° hue adjustment), and noise injection (Gaussian blur up to 2px, sensor noise up to 1.57% of pixels). Each image generated 3 augmented variants, yielding a final training dataset of 1,416 images. The YOLOv8 nano variant was selected for its balance of inference speed and accuracy-critical for real-time drone navigation where processing latency directly impacts flight safety and navigation precision.

The model was trained iteratively in batches, with performance evaluation after each batch informing the composition of subsequent training data. This approach enabled targeted refinement in challenging detection scenarios, such as overlapping poles or partially occluded objects. Training was conducted using PyTorch on NVIDIA RTX hardware with GPU acceleration, reducing training time to approximately 25 minutes per iteration.

The detection module generates structured snapshots containing object labels, confidence scores, bounding box coordinates, and per-frame summary statistics. Each snapshot is timestamped and includes frame rate information, enabling the system to assess perception latency and detection freshness. The zero-shot capabilities of the base YOLO model were augmented through fine-tuning on domain-specific objects such as poles to improve reliability in our operational environment.

Additionally, the OpenCV function for QR code detection was implemented as waypoints that represent last known

GPS pings of a lost person. The QR code corners were appended per frame to the perception blackboard such that the LLM had access to positions at all time and could orient towards it.

1. Multi-Camera Configuration

The DJI Tello platform provides both forward-facing and downward-facing cameras, which are exploited for different mission phases. The forward camera is used during obstacle navigation and target approach, while the downward camera enables precision overhead positioning and radial search operations. Camera switching is managed programmatically through tool calls, with the LLM determining appropriate camera selection based on the current sub-task requirements.

IV. Conclusion

This work validates our LLM-enabled planning framework, LEMUR, as a viable approach for autonomous UAV missions that require adaptive decision-making under uncertainty. Our results demonstrate a clear trade-off: while traditional state machines excel at time-critical reactive control, such as obstacle avoidance (completing it twice as fast), LLM agents are superior at high-level sequential tasks. This was evident in the search phase, which finished faster using adaptive patterns, and the return-to-base phase, which showed less variability due to robust planning and failure recovery.

The primary cost of this strategic advantage is a planning latency, which caused the agent to remain stationary for 40% of its obstacle traversal time. While inefficient for purely reactive maneuvers, this overhead is acceptable within the mission’s overall timescale. We conclude that a hybrid approach, combining the millisecond-level control of state machines with the strategic robustness of an LLM, offers the most promising path forward, enhancing both the speed and reliability of search-and-rescue missions.

Future work should scale this concept to multi-agent swarms, where a central LLM planner directs sub-agents that execute low-latency local controllers. This would enable sophisticated, coordinated actions while maintaining real-time resilience. Overall, LEMUR demonstrates a practical pathway to scalable, human-supervised autonomous systems that effectively blend reactive control with adaptive, failure-aware planning.

Acknowledgments

We thank Dr Zihao Wang for his support, advice and access to resources.

References

- [1] Mon-Williams, R., Li, G., Long, R., Du, W., and Lucas, C. G., “Embodied large language models enable robots to complete complex tasks in unpredictable environments,” *Nature Machine Intelligence*, Vol. 7, 2025. <https://doi.org/10.1038/s42256-025-01005-x>, URL <https://www.nature.com/articles/s42256-025-01005-x>.
- [2] Cao, Y., Hong, S., Li, X., Ying, J., Ma, Y., Liang, H., Liu, Y., Yao, Z., Wang, X., Huang, D., Zhang, W., Huang, L., Chen, M., Hou, L., Sun, Q., Ma, X., Wu, Z., Kan, M.-Y., Lo, D., Zhang, Q., Ji, H., Jiang, J., Li, J., Sun, A., Huang, X., Chua, T.-S., and Jiang, Y.-G., “Toward Generalizable Evaluation in the LLM Era: A Survey Beyond Benchmarks,” , 2025. URL <https://arxiv.org/abs/2504.18838>.
- [3] Huang, W., Wang, C., Zhang, R., Li, Y., Wu, J., and Fei-Fei, L., “VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models,” , 2023. URL <https://arxiv.org/abs/2307.05973>.
- [4] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., Ibarz, J., Ichter, B., Irpan, A., Jackson, T., Jesmonth, S., Joshi, N. J., Julian, R., Kalashnikov, D., Kuang, Y., Leal, I., Lee, K.-H., Levine, S., Lu, Y., Malla, U., Manjunath, D., Mordatch, I., Nachum, O., Parada, C., Peralta, J., Perez, E., Pertsch, K., Quiambao, J., Rao, K., Ryoo, M., Salazar, G., Sanketi, P., Sayed, K., Singh, J., Sontakke, S., Stone, A., Tan, C., Tran, H., Vanhoucke, V., Vega, S., Vuong, Q., Xia, F., Xiao, T., Xu, P., Xu, S., Yu, T., and Zitkovich, B., “RT-1: Robotics Transformer for Real-World Control at Scale,” , 2023. URL <https://arxiv.org/abs/2212.06817>.
- [5] Liang, J., Huang, W., Xia, F., Xu, P., Hausman, K., Ichter, B., Florence, P., and Zeng, A., “Code as Policies: Language Model Programs for Embodied Control,” , 2023. URL <https://arxiv.org/abs/2209.07753>.
- [6] Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., and Sachdeva, N., “Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities,” , 2025. URL <https://arxiv.org/abs/2507.06261>.